



Cyberscope

A *TAC Security* Company

Audit Report

VoltPayV2

October 2026

Network BSC

Address 0xE90714e6e4becEc65F33D3099F95B42B6e3168aE

Audited by © cyberscope

Analysis

● Critical ● Medium ● Minor / Informative ● Pass

Severity	Code	Description	Status
●	ST	Stops Transactions	Passed
●	OTUT	Transfers User's Tokens	Passed
●	ELFM	Exceeds Fees Limit	Passed
●	MT	Mints Tokens	Passed
●	BT	Burns Tokens	Passed
●	BC	Blacklists Addresses	Passed

Diagnostics

● Critical ● Medium ● Minor / Informative

Severity	Code	Description	Status
●	ILSE	Initial Liquidity Seeding Exposure	Acknowledged
●	APD	Arbitrary Pair Designation	Acknowledged
●	CCR	Contract Centralization Risk	Acknowledged
●	MCM	Misleading Comment Messages	Acknowledged
●	PPFE	Predicted Pair Fee Exemption	Acknowledged
●	RRA	Redundant Repeated Approvals	Acknowledged
●	USPL	Unexempted Secondary Pair Limits	Acknowledged
●	UPFB	Unrecognized Pool Fee Bypass	Acknowledged

Table of Contents

Analysis	1
Diagnostics	2
Table of Contents	3
Risk Classification	4
Review	5
Audit Updates	5
Source Files	5
Findings Breakdown	6
ILSE - Initial Liquidity Seeding Exposure	7
Description	7
Recommendation	7
APD - Arbitrary Pair Designation	9
Description	9
Recommendation	9
CCR - Contract Centralization Risk	10
Description	10
Recommendation	10
MCM - Misleading Comment Messages	11
Description	11
Recommendation	11
PPFE - Predicted Pair Fee Exemption	12
Description	12
Recommendation	12
RRA - Redundant Repeated Approvals	13
Description	13
Recommendation	13
USPL - Unexempted Secondary Pair Limits	14
Description	14
Recommendation	14
UPFB - Unrecognized Pool Fee Bypass	15
Description	15
Recommendation	15
Functions Analysis	16
Inheritance Graph	18
Flow Graph	19
Summary	20
Disclaimer	21
About Cyberscope	22

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation:** This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation:** This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical:** Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium:** Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor:** Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative:** Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
● Critical	Highly Likely / High Impact
● Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
● Minor / Informative	Unlikely / Low to no Impact

Review

Contract Name	VoltPayV2
Compiler Version	v0.8.35+commit.47b9dedd
Optimization	200 runs
Explorer	https://bscscan.com/address/0xe90714e6e4becec65f33d3099f95b42b6e3168ae
Address	0xe90714e6e4becec65f33d3099f95b42b6e3168ae
Network	BSC
Symbol	VLT
Decimals	18
Total Supply	200,000,000

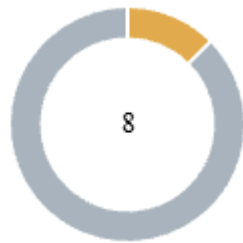
Audit Updates

Initial Audit	02 Oct 2026
---------------	-------------

Source Files

Filename	SHA256
project/mainnet-contracts/VoltPayV2_FINAL_0_8_35.sol	9d708e7e874bdf72533d1f59d976049dfee ff9f3555aeccfd4c2bfdc94cbcdade

Findings Breakdown



● Critical	0
● Medium	1
● Minor / Informative	7

Severity	Unresolved	Acknowledged	Resolved	Other
● Critical	0	0	0	0
● Medium	0	1	0	0
● Minor / Informative	0	7	0	0

ILSE - Initial Liquidity Seeding Exposure

Criticality	Medium
Location	VoltPayV2_FINAL_0_8_35.sol#L219,475
Status	Acknowledged

Description

The primary pair is created in the constructor, and the deployed pair currently holds no liquidity. Any address can send 1 wei of WBNB to the pair and call `sync()`, after which the router rejects every `addLiquidityETH` call with `INSUFFICIENT_LIQUIDITY`.

The workaround is to send VLT to the pair directly and mint in a later transaction. Between those two steps, any user can add a small amount of WBNB and call `mint()` to receive the LP tokens backed by the owner's VLT. LP minting moves no VLT, so the trading gate does not apply. After trading is enabled the attacker burns the LP and receives the tokens.

```
        address createdPair =
        IUniswapV2Factory(factoryAddress).createPair(
            address(this),
            wrappedBNB
        );
        ...
        if (!tradingEnabled) {
            require(
                from == owner() ||
                to == owner() ||
                from == address(this) ||
                to == address(this),
                "trading not active"
            );
        }
```

Recommendation

The team is advised to ensure the initial liquidity is provided in a single operation that third parties cannot interleave. This will prevent the launch from being blocked and the seeded tokens from being captured.

Team Update

This issue will be resolved once the PinkSale presale is completed and the liquidity is created through the PinkSale finalization process.

APD - Arbitrary Pair Designation

Criticality	Minor / Informative
Location	VoltPayV2_FINAL_0_8_35.sol#L286,297,497
Status	Acknowledged

Description

Any non-zero address can be designated as a pair by the owner, including user wallets. Transfers to that user are treated as sells, charged 4% and capped at `maxTx` while limits are active. Transfers from that user are treated as buys, charged 2% and checked against the recipient's `maxTx` and `maxWallet`. The user's own sells to the pair pay 2% instead of 4%.

```
isPair[pairAddress] = status;
isLimitExempt[pairAddress] = status;
...
if (isSell && !isLimitExempt[from]) {
    require(amount <= maxTx, "sell limit");
}
```

Recommendation

The team is advised to restrict pair designation to actual AMM pools and to document any address designated as a pair. This will prevent user wallets from being taxed or limited as pairs.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	VoltPayV2_FINAL_0_8_35.sol
Status	Acknowledged

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

```
function enableTrading() external onlyOwner {
    function setPair(address pairAddress, bool status) external onlyOwner
    {
        function setExemptions(
            address account,
            bool feeExempt,
            bool limitExempt
        ) external onlyOwner {
            function setSwapSettings(uint256 threshold, uint256 maxAmount)
            external onlyOwner {
                function rescueBNB() external nonReentrant onlyOwner {
                function rescueTokens(address tokenAddress) external nonReentrant
                onlyOwner {
```

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

MCM - Misleading Comment Messages

Criticality	Minor / Informative
Location	VoltPayV2_FINAL_0_8_35.sol#L190,568,579
Status	Acknowledged

Description

The contract is using misleading comment messages. These comment messages do not accurately reflect the actual implementation, making it difficult to understand the source code. As a result, the users will not comprehend the source code's actual implementation.

```
``solidity
    event TreasuryTransferFailed(uint256 amount);
...
    // For a 50/50 overall fee split:
    // 25% of tokens remain as the token side of LP,
    // 75% are swapped; one third of received BNB funds the BNB side
of LP.
...
    // Pull-payment accounting: _swap never calls the treasury.
    // This removes an untrusted external native-coin call from
transfer flow.
```

'''

Recommendation

The team is advised to carefully review the comment in order to reflect the actual implementation. To improve code readability, the team should use more specific and descriptive comment messages.

PPFE - Predicted Pair Fee Exemption

Criticality	Minor / Informative
Location	VoltPayV2_FINAL_0_8_35.sol#L314,454
Status	Acknowledged

Description

PancakeSwap pair addresses are deterministic, while `setExemptions` only rejects addresses that already carry pair bytecode. The owner can therefore exempt the address of a future VLT pair before it is created. Once that pair is deployed, every buy and sell through it skips the fee, contradicting the comment that a trading pair must never bypass fees.

```
require(!configFinalized, "Config finalized");  
require(account != address(0), "Account zero");  
require(!_isAmmPair(account), "Pair exemptions forbidden");
```

Recommendation

The team is advised to ensure that addresses recognized as pairs at transfer time cannot carry fee exemptions, regardless of when the exemption was set. This will keep the pair fee guarantee intact.

RRA - Redundant Repeated Approvals

Criticality	Minor / Informative
Location	VoltPayV2_FINAL_0_8_35.sol#L612,623,633,649
Status	Acknowledged

Description

The contract is designed to `approve` token transfers during the contract's operation by calling the `_approve` function before specific operations. This approach results in additional gas costs since the approval process is repeated for every operation execution, leading to inefficiencies and increased transaction expenses.

```
_approve(address(this), address(router), tokenAmount);

tokensUsed, nativeUsed, liquidityMinted) = router.addLiquidityETH{value:
nativeAmount}(
...
    _approve(address(this), address(router), 0);
...
    _approve(address(this), address(router), tokenAmount);
...
    router.swapExactTokensForETHSupportingFeeOnTransferTokens(
...
    _approve(address(this), address(router), 0);
```

Recommendation

Since the approved address is a trusted third-party source, it is recommended to optimize the contract by approving the maximum amount of tokens once in the initial set of the variable, rather than before each operation. This change will reduce the overall gas consumption and improve the efficiency of the contract.

USPL - Unexempted Secondary Pair Limits

Criticality	Minor / Informative
Location	VoltPayV2_FINAL_0_8_35.sol#L241,493
Status	Acknowledged

Description

Only the primary pair and pairs registered through `setPair` are limit exempt. Secondary VLT pairs recognized by bytecode are not, and pairs created after `finalizeConfig` cannot be registered. Multi-hop swaps send tokens directly from one pair to the next, so a route through such a pair hits the wallet limit check against that pair's balance and reverts while limits are active.

```
isLimitExempt[createdPair] = true;  
...  
require(balanceOf(to) + amount <= maxWallet, "wallet limit");
```

Recommendation

The team is advised to ensure that all VLT pairs are treated consistently by the launch limits, or to remove the limits before secondary pairs are used for routing.

UPFB - Unrecognized Pool Fee Bypass

Criticality	Minor / Informative
Location	VoltPayV2_FINAL_0_8_35.sol#L122,453,485
Status	Acknowledged

Description

Fees and limits apply only when one side of a transfer is recognized as a pair. Recognition requires an `isPair` entry or runtime bytecode identical to the primary PancakeSwap V2 pair, and that reference hash is immutable. Pools on PancakeSwap V3 or V2 forks with different bytecode, such as `BiSwap`, are never detected.

Once trading is enabled, any holder can create such a pool and add liquidity. Trades against that pool are processed as plain wallet transfers, so they pay no fee and skip `maxTx` and `maxWallet`. Registering a V3 pool through `setPair` does not help, because the sell fee makes the pool's balance check fail on every sell.

```
bytes32 public immutable primaryPairCodeHash;
...
function _isAmmPair(address account) internal view returns (bool) {
    return isPair[account] || account.codehash ==
primaryPairCodeHash;
}
```

Recommendation

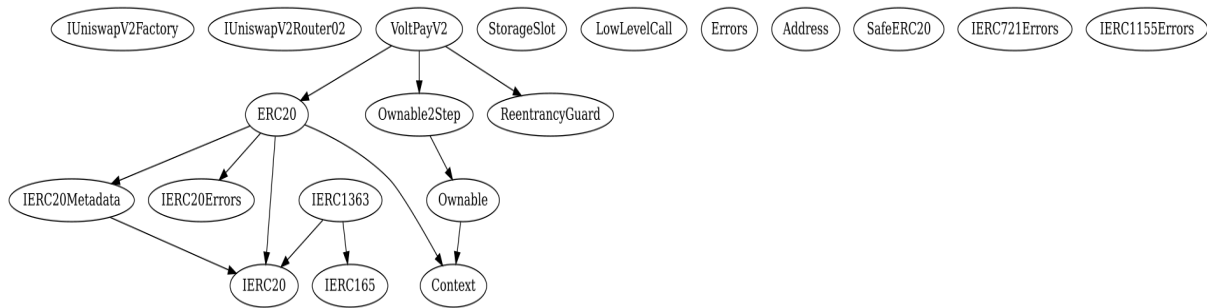
The team is advised to evaluate which trading venues the fee and limit model is expected to cover and to document that trades on other venues are untaxed. This will set accurate expectations for holders and for the fee revenue.

Functions Analysis

Contract	Type	Bases		
	Function Name	Visibility	Mutability	Modifiers
IUniswapV2Factory	Interface			
	createPair	External	✓	-
IUniswapV2Router02	Interface			
	factory	External		-
	WETH	External		-
	addLiquidityETH	External	Payable	-
	swapExactTokensForETHSupportingFeeOnTransferTokens	External	✓	-
VoltPayV2	Implementation	ERC20, Ownable2Step, ReentrancyGuard		
		Public	✓	ERC20 Ownable
	enableTrading	External	✓	onlyOwner
	removeLimits	External	✓	onlyOwner
	increaseLimits	External	✓	onlyOwner
	setPair	External	✓	onlyOwner
	setExemptions	External	✓	onlyOwner

	finalizeConfig	External	✓	onlyOwner
	reduceFees	External	✓	onlyOwner
	lockFees	External	✓	onlyOwner
	lockSwapSettings	External	✓	onlyOwner
	setSwapSettings	External	✓	onlyOwner
	renounceOwnership	Public	✓	onlyOwner
	rescueBNB	External	✓	nonReentrant onlyOwner
	rescueTokens	External	✓	nonReentrant onlyOwner
	retryTreasuryPayment	External	✓	nonReentrant onlyFeeProcess or
	_isAmmPair	Internal		
	_update	Internal	✓	
	processFees	External	✓	nonReentrant onlyFeeProcess or lockSwap
	_swap	Internal	✓	
	_addLiquidity	Internal	✓	
	_swapTokensForNative	Internal	✓	
		External	Payable	-

Inheritance Graph



[illegible]

Summary

VoltPayV2 contract implements a token mechanism. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security. Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.



A **TAC Security** Company

The Cyberscope team

cyberscope.io